



Automated Compliance for Embedded C.

Getting Started with ComplyC

Beta Release Quick Start Guide

Scan an Embedded C Project - Review Violations - Improve Code

Comply C - Automatic Compliance Tool

Beta Release for Community Review

Engineering Procedure

Doc. Revision: 1.0

Release Date: 17Aug26

Status: *Released*

Prepared By:

Kishore Gorijavolu

1. What ComplyC Does

ComplyC is an open-source static-analysis and coding-guideline review tool focused on C and embedded-software projects. It analyzes source code against the enabled rule profile and generates reviewable results, including HTML/JSON reports.

- Coding-style and naming checks
- Function size, parameter count, complexity and nesting checks
- Control-flow and dead-code checks
- Selected safety-oriented checks
- Project-oriented source scanning and report generation

Beta note

ComplyC is a community beta. It is intended for engineering review and feedback, not as proof of MISRA, CERT, ISO 26262, ASPICE, cybersecurity, or regulatory compliance.

2. Before You Start

For the smoothest first evaluation, use a small C project or a representative module before scanning a very large production repository.

- Windows PC for the packaged beta executable
- A C or embedded-C project containing .c/.h files
- GCC available when preprocessing is required by the selected workflow
- Permission to read the selected project directory

For embedded projects, vendor headers, generated files, compiler-specific macros, and conditional compilation can affect preprocessing. The beta contains preprocessing-recovery logic, but unusual toolchains may still require project-specific handling.

3. Start ComplyC

Launch the ComplyC application from the beta package. If Windows displays a security warning for an unsigned community-beta executable, verify that the file came from the official ComplyC release before continuing.

Locate the **dist** folder

Name	Date modified	Type	Size
complyc	8/14/2026 3:52 PM	File folder	
dist	8/17/2026 2:16 PM	File folder	
examples	8/14/2026 3:52 PM	File folder	
fake_libc_include	8/17/2026 2:24 PM	File folder	
rules	8/14/2026 3:52 PM	File folder	
tests	8/14/2026 3:52 PM	File folder	
BUILD_INSTRUCTIONS.md	8/14/2026 3:52 PM	Markdown Source...	2 KB
build_windows_exe.bat	8/14/2026 3:52 PM	Windows Batch File	1 KB
complyc_gui.py	8/14/2026 3:52 PM	Python Source File	43 KB
complyc_scan_worker.py	8/14/2026 3:52 PM	Python Source File	7 KB
ComplyC-GUI.spec	8/14/2026 3:52 PM	SPEC File	1 KB
requirements.txt	8/14/2026 3:52 PM	TXT File	1 KB
RUN_FROM_SOURCE.bat	8/14/2026 3:52 PM	Windows Batch File	1 KB
STABLE_BUILD_NOTES.md	8/14/2026 3:52 PM	Markdown Source...	1 KB
STABLE_VERIFICATION_v0_9_0-beta.md	8/14/2026 3:52 PM	Markdown Source...	2 KB

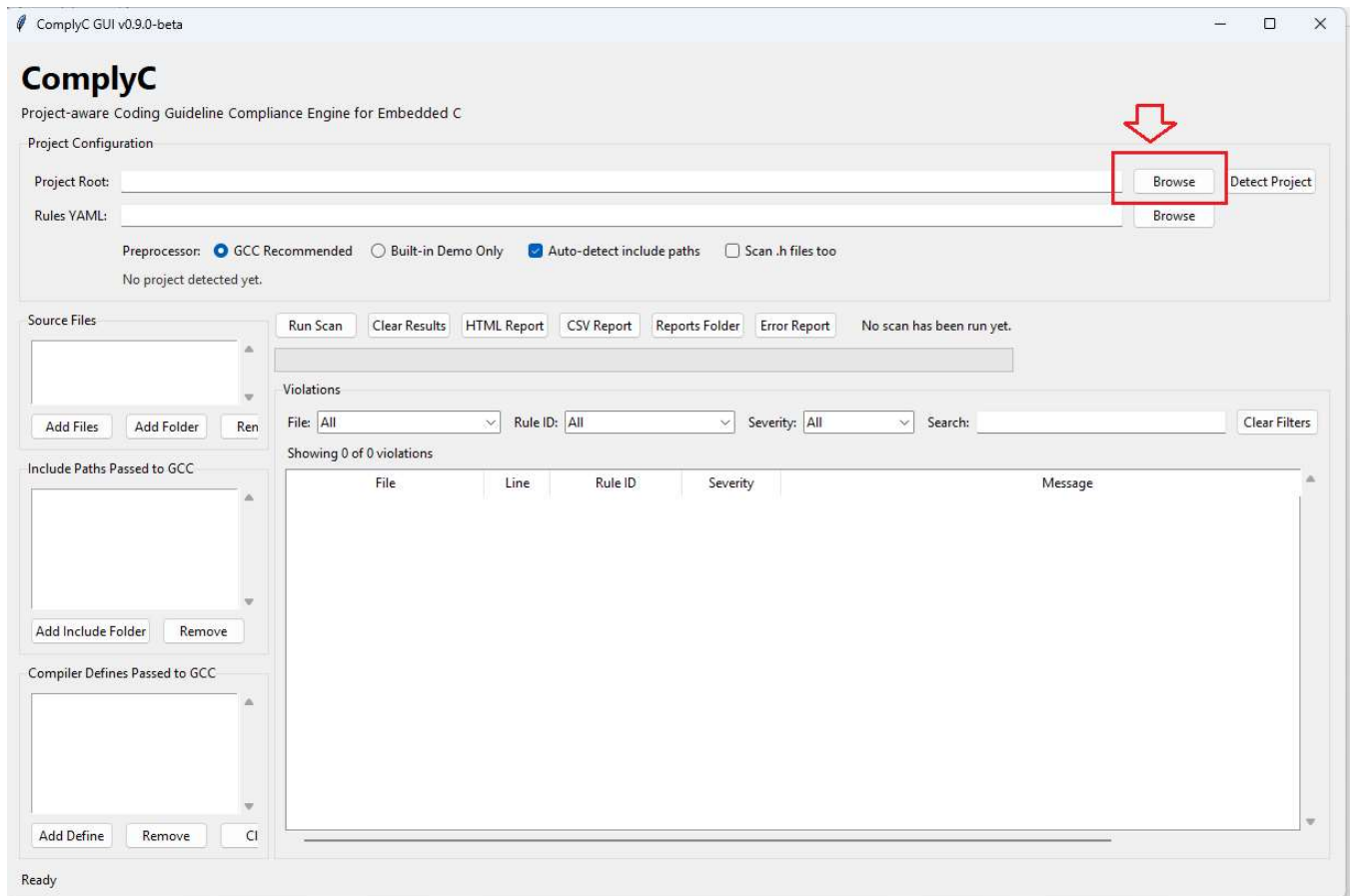
Inside the dist folder, you will find **ComplyC-GUI.exe**

Double-click on the exe file

It will open the GUI Tool to run the coding guide checks.

4. Select Your Project

- Choose the root folder of the C project you want to analyze.



- Confirm that ComplyC detects the expected source structure. You will find the compiler details that the project is using once ComplyC detects the compiler and source code automatically.
- Then select the Rules so that the code will be checked against them.
- The default rule engine is located in the rule folder of the ComplyC folder under the name **complyc_style.yml**
- You may customize the rules by commenting on the rules (disabling them) or enabling all checks designed for the project.
- You can also remove some source files that are not developed by the engineer by clicking on them and selecting Remove.
- Add any macro definitions, compiler defines passed to the project, and project custom header file locations manually to the tool left-hand side sections.
- Once the configuration is complete, you can hit **Run Scan**.
- For a first run, prefer a project/module that builds successfully in its normal development environment.
- If include paths are discovered automatically, review them before scanning when the project uses vendor SDKs or non-standard folders.

Recommended first test

Start with a small, known module. This makes it easier to distinguish genuine violations from preprocessing or project-configuration issues.

5. Review of the Coding Rule Profile

ComplyC uses a configurable coding-style/rule profile. Review the supplied YAML configuration before treating a reported item as a project requirement.

- Enable or disable rules are appropriate for the evaluation.
- Review configurable thresholds such as function length, complexity, nesting depth, and parameter count.
- Review naming conventions before scanning an existing codebase.
- Keep a copy of the profile used for the scan so results can be reproduced.

Important

A ComplyC violation means the source matched an enabled rule. It does not automatically mean the software contains a functional defect.

6. Run the Scan

Start the analysis from the GUI after selecting the project and reviewing the configuration. ComplyC discovers relevant source files, performs preprocessing where required, runs enabled checks, and builds the report.

During beta testing, pay attention to:

- Files that are skipped
- Preprocessor/include errors
- Unexpected source-line mapping
- False positives or false negatives
- Incorrect classification of static/global objects
- Unexpected behavior around `#if/#ifdef` code

7. Review Violations

Review each reported violation in context. Use the rule identifier, source file, source line, and message to determine whether the item is a valid coding-guideline violation.

- Filter results by rule or violation type when useful.
- Use source navigation/double-click behavior when available to inspect the affected code.
- Correct genuine violations and re-run the scan.
- Record suspected false positives with a minimal source example whenever possible.

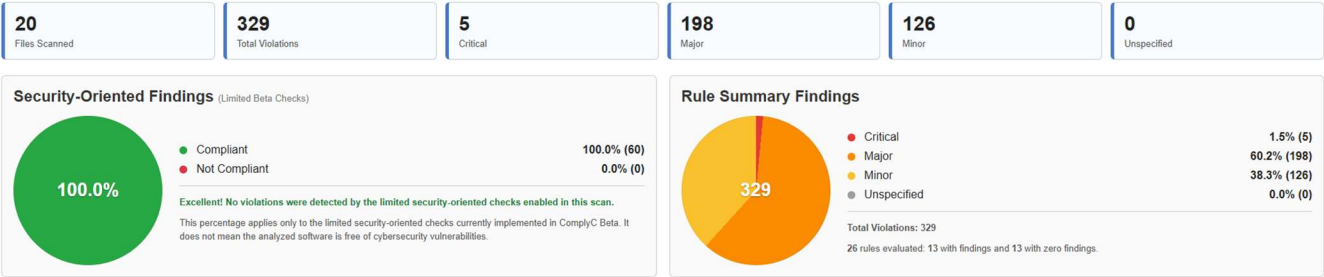
Typical beta checks include naming conventions, mandatory braces, function size, parameter count, cyclomatic complexity, nesting depth, switch defaults, fall-through, empty blocks/statements, unreachable code, recursion, dynamic memory, goto, infinite-loop patterns, and selected unsafe C APIs.

8. Read the HTML Report

The HTML report is the easiest artifact to share during beta evaluation. Use it to review scan findings and the rule/security summaries generated by the current release. You can review the report by clicking **HTML report button**

ComplyC – Coding Style Report

Dashboard



9. Interpreting Common Results

Finding	What It Usually Means	What To Do
Naming violation	Identifier does not match the enabled naming convention.	Rename it or adjust the project profile if the convention is not applicable.
Complexity/nesting	Function exceeds the configured maintainability threshold.	Review whether the logic can be simplified or decomposed.
Switch/default	Switch structure violates an enabled control-flow rule.	Add explicit handling or document/configure an intentional exception.
Unsafe API	Source uses an API flagged by the selected safety profile.	Review buffer bounds and use an approved interface where appropriate.
Preprocessing error	ComplyC could not fully reproduce the project's preprocessing environment.	Check includes paths, compiler macros, vendor headers, and the scan error report.
Unexpected line/file	Source mapping may need review.	Capture the report and a minimal reproducer for the beta issue.

10. If the Scan Fails

Do not immediately change production source code to work around a ComplyC parsing or preprocessing problem.

- Check the generated scan/error report first.
- The Error Reports button will take you to the skipped files and the reasoning behind them.
- Confirm that the required directories are available.
- Check whether the source depends on compiler-specific predefined macros.
- Check whether an #error branch is selected because the expected compiler macro is absent.
- Try a smaller source module to isolate the failure.
- Report reproducible beta defects rather than hiding them with source changes.

11. What Feedback Is Most Useful

Community feedback is most useful when another developer can reproduce the observation.

- ComplyC version
- Operating system
- Rule ID, if applicable
- Small source snippet or minimal sample project
- Expected result
- Actual result
- Relevant HTML/error report
- Steps needed to reproduce the issue

Privacy

Do not upload proprietary employer/customer source code, credentials, confidential configuration, or other restricted material to a public issue.

12. Suggested 15-Minute Beta Evaluation

1. Launch ComplyC.
2. Select a small C project.
3. Review the supplied coding-rule configuration.
4. Run the scan.
5. Open the generated HTML report.
6. Inspect five to ten reported findings.
7. Verify source file and line mapping.
8. Correct one simple violation and scan again.
9. Record any false positive, false negative, crash, or preprocessing problem.
10. Share concise feedback through the project's official issue/review channel.

13. Quick Troubleshooting

Problem	First Check
Project is not detected	Select the actual project/root source directory and verify source files are present.
Header not found	Verify vendor/project include directories.
Compiler not supported / #error	Check compiler-selection macros used by the project.
Too many unexpected violations	Review the active YAML profile and thresholds.
Wrong source line	Preserve the report and provide a minimal reproducer.
GUI appears stuck	Allow the current scan to complete; for a repeatable hang, capture the project size and last visible operation and report it.

14. Beta Release Checklist for Users

- ☐ I obtained ComplyC from the official release source.
- ☐ I started with non-confidential or authorized source code.
- ☐ I reviewed the enabled rule profile.
- ☐ I completed at least one scan.
- ☐ I reviewed the HTML report.
- ☐ I verified several findings against original source code.
- ☐ I captured reproducible information for any issue I found.

15. Next Steps

After the first successful scan, read the ComplyC Embedded C Coding Guidelines for explanations of the coding practices represented by the beta rule set. Then adapt the rule profile to the conventions appropriate for your project and repeat the scan.

For source code, releases, issues, and project updates, use the official ComplyC repository:

github.com/Kishore-Gorijavolu/ComplyC

Beta Disclaimer

ComplyC is beta software. Results require engineering review. The tool and this guide do not constitute certification, functional-safety approval, cybersecurity certification, or evidence of compliance with MISRA, CERT, ISO 26262, ASPICE, or any other third-party standard.

Document Change History

Revision	Description	Author(s)	Revision Date	Approved By (Title)	Approved By (Name)	Approved Date
1.0	Initial Version Released for Community Review	Kishore Gorijavolu	17Aug2026	N/A	N/A	N/A